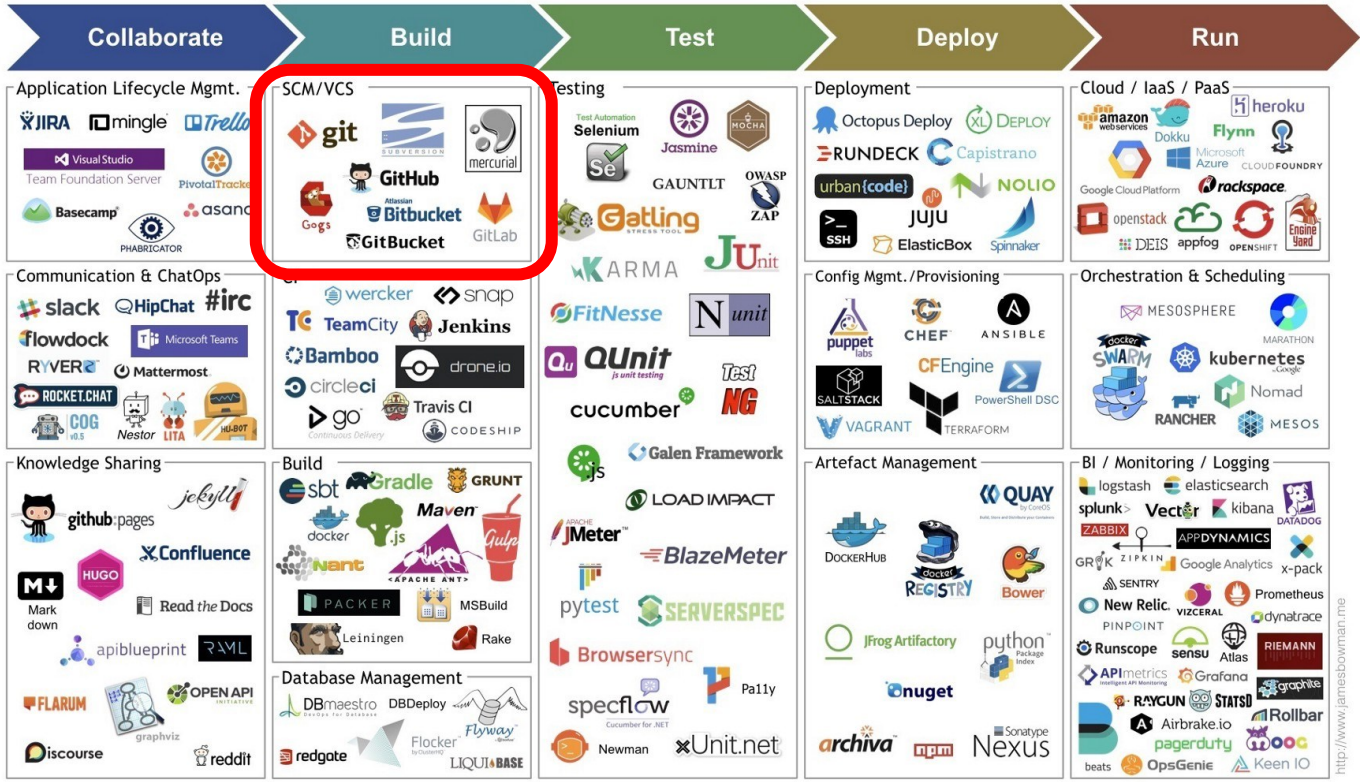
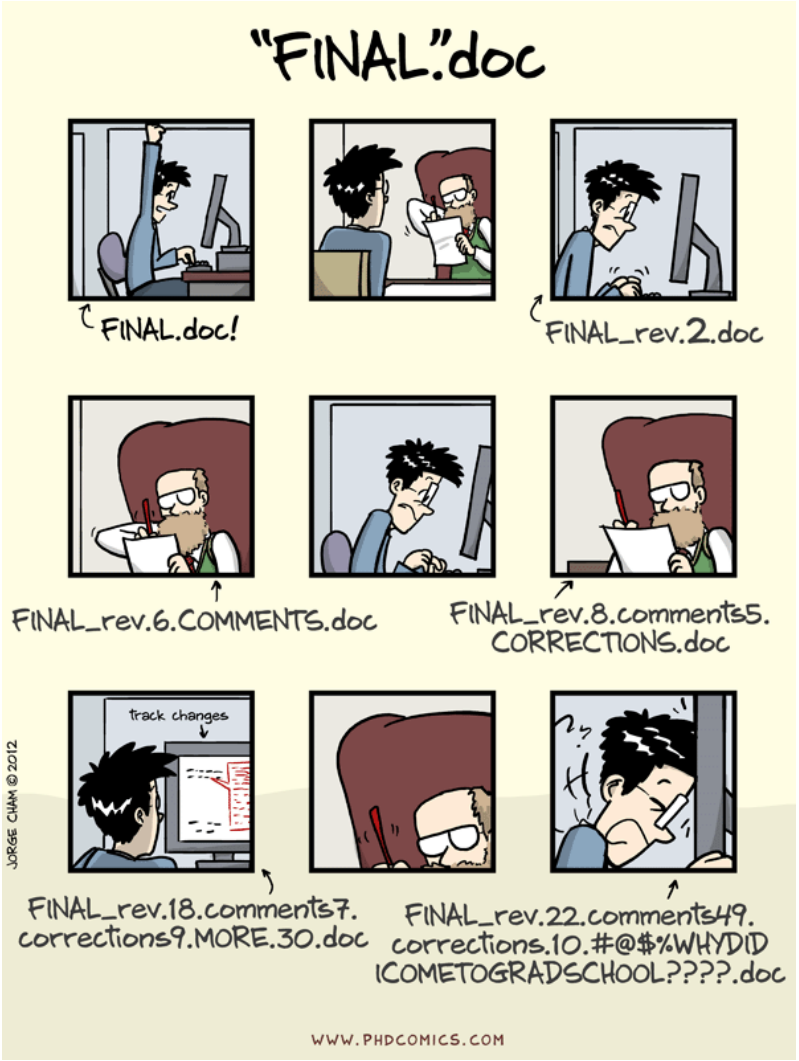
Collaboration and Version Control in Software Development

The need for Version Control

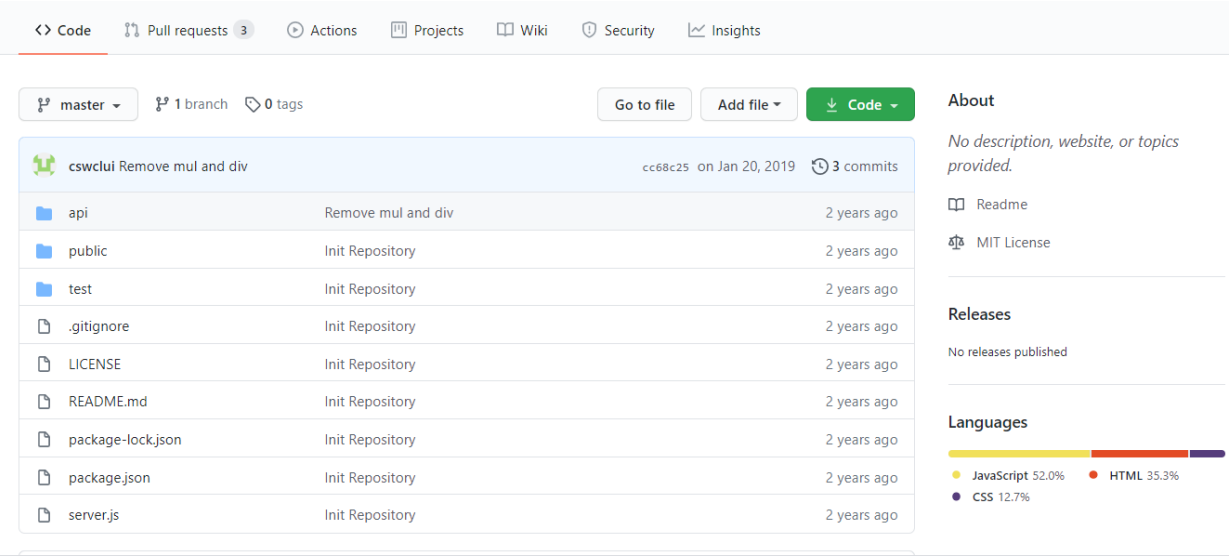
- "Piled Higher and Deeper" by Jorge Cham
  - http://www.phdcomics.com





Git and GitHub

- **Git** is a version control system that lets you manage and keep track of your source code history
  - What is Git? Explained in 2 Minutes!
    - https://www.youtube.com/watch?v=2ReR1YJrNOM
- **Git Repository** is a container for your entire project
  - Every file or subfolder within it belongs to that repository
- **GitHub** is a cloud-based hosting service that lets you manage **Git** repositories.







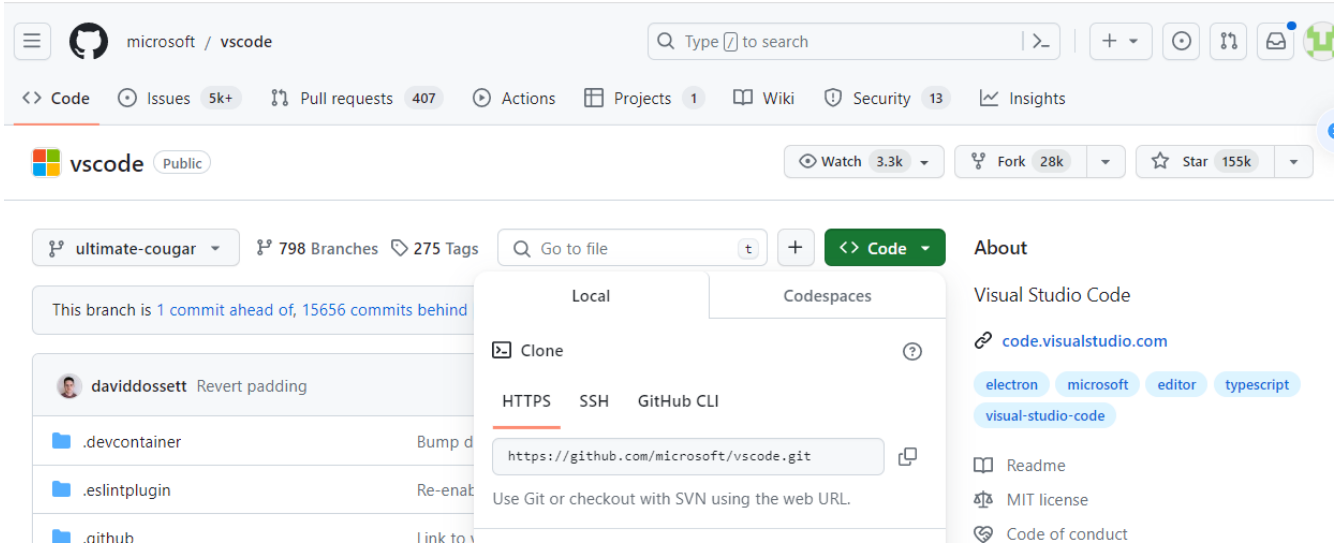
Azure Repos





Git Repositories

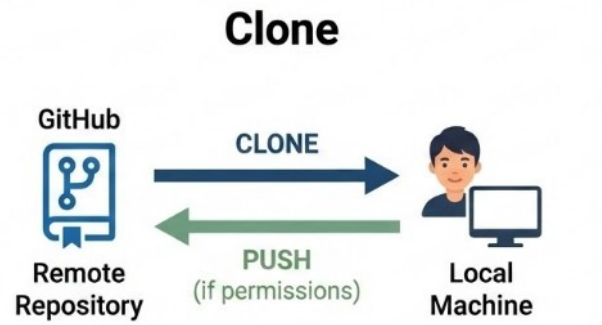
- A repository is where your project work happens
- Think of it as your project folder. It contains all of your project's files and revision history.
- You can work within a repository alone or invite others to collaborate with you on those files.



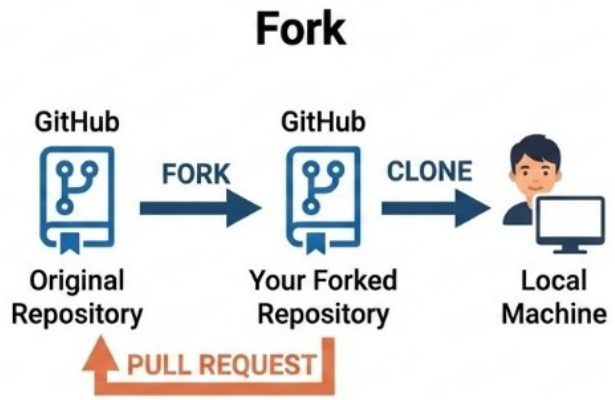
https://github.com/microsoft/vscode

Clone vs. Fork

- **Cloning**: Create a local copy on your machine for software development
  - Pulls down all the repository data that GitHub has at that point in time, including all versions of every file and folder for the project!
  - Fix issues, add or remove files, experiment with your project



- **Forking**: Create a personal copy of the repository on GitHub
  - You may develop on your fork and submit changes to the original repository via pull request
  - Very popular when contributing to open source software projects



Git commands



Git Commands	Purpose
git clone	Clones an existing repository, creating a local copy with its history.
git add	Moves changes from the working directory to the staging area.
git commit	Commits changes from the staging area to the local repository.
git push	Pushes local commits to the remote repository, synchronizing repositories.
git pull	Fetches updates from the remote repository into the local repository.
git fetch	Retrieves the latest updates from the remote repository.
git merge	Integrates fetched updates into the current branch.
git checkout	Switches branches or restores files from different commits or branches.
git log	Shows the commit history, including commit messages, authors, and timestamps.
git status	Displays the current status of the repository, showing changes and branch information.

Project folder setup

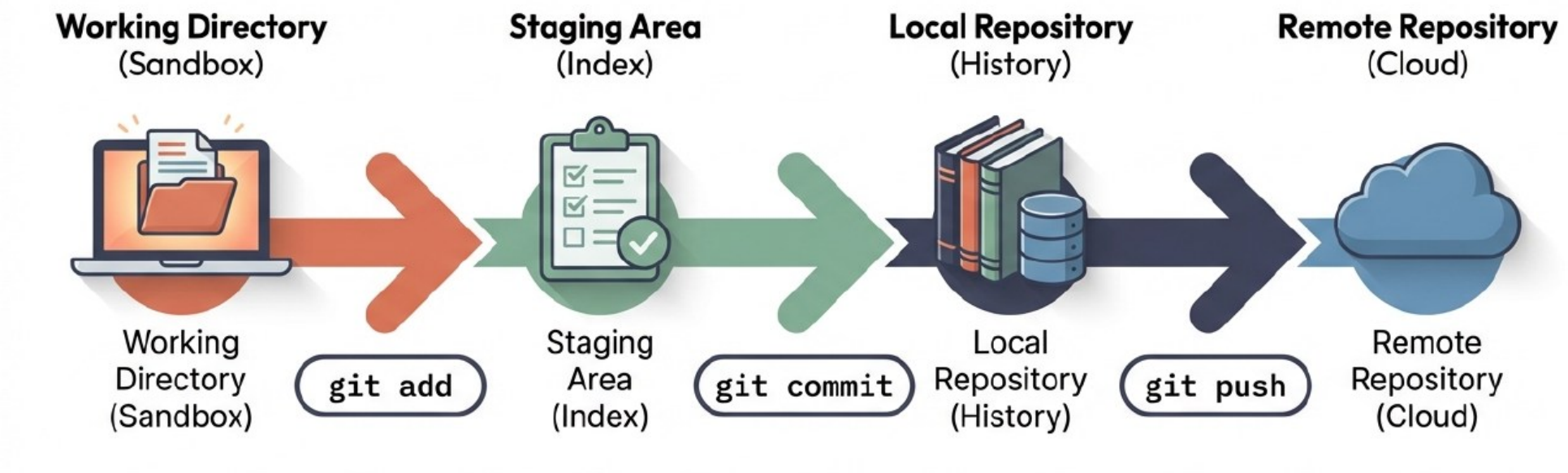
- **git clone <url>:** Creates a local copy and pulls down all repository data, including every version of every file that has ever existed.
- **git init**: Turns a current local directory into a new, empty Git repository.
- **.git** folder: stores the repository's history



Remote repository:
A server hosting your Git repository,
e.g. GitHub or Bitbucket.

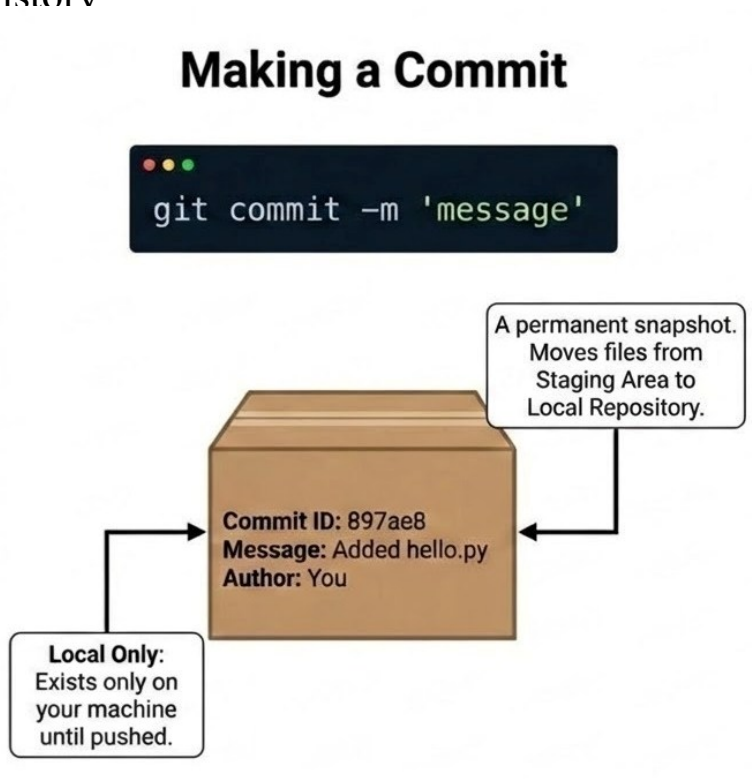
Crucial zones in Git

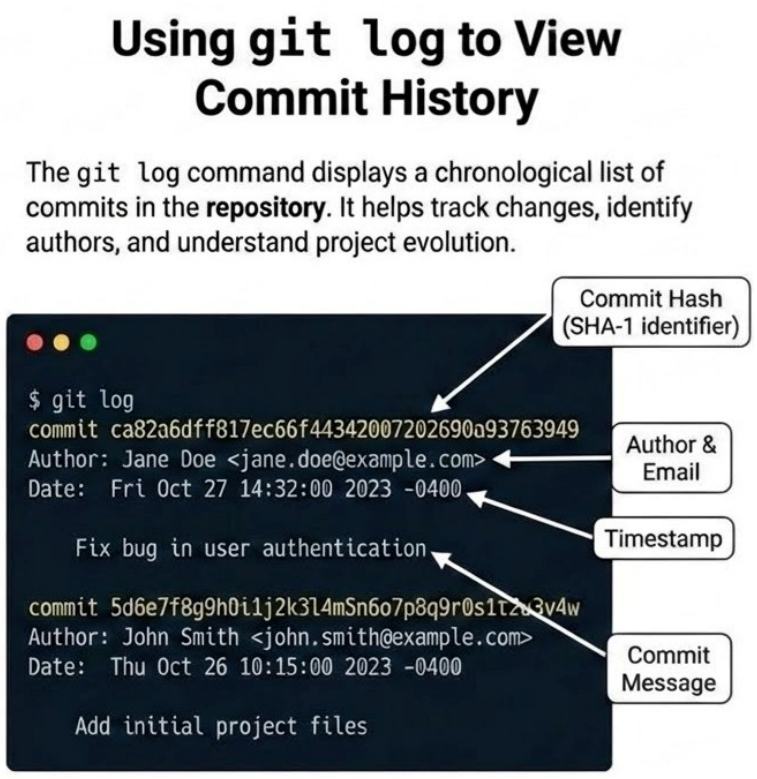
- **Working Directory**: The actual folder on your computer's filesystem where you are currently creating, editing, and deleting files
- **Staging Area:** "Loading Dock" or a middle ground between your workspace and your permanent history
- **Local repository:** Permanent records of your project at specific points in time on your local computer.



Commit

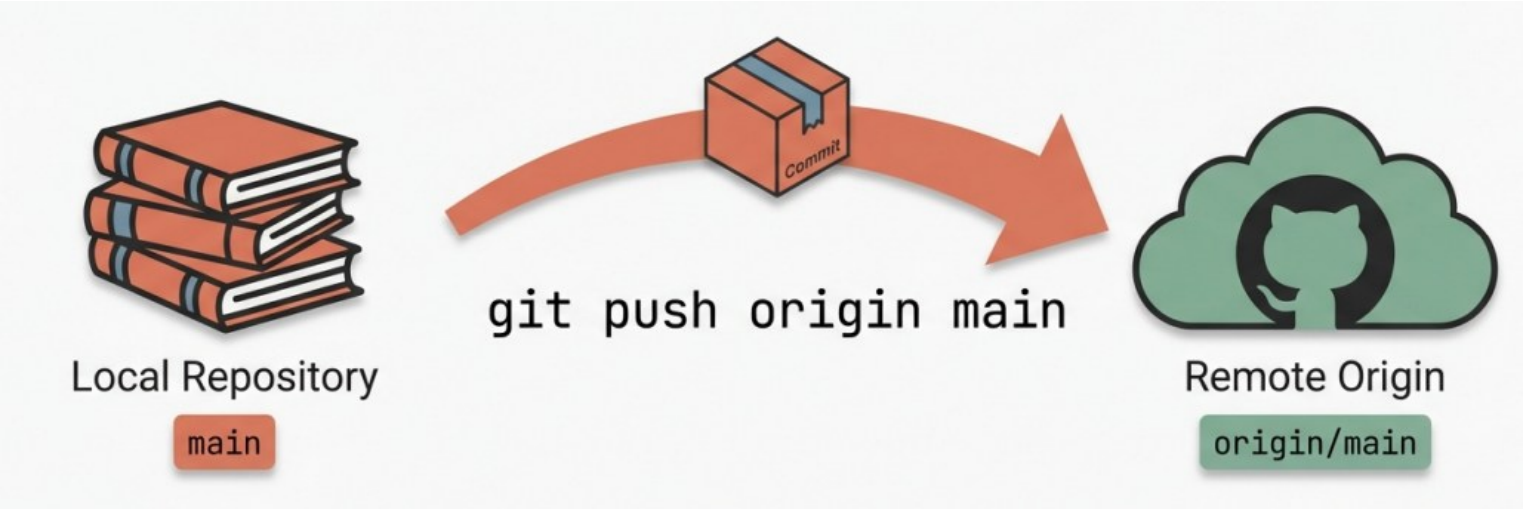
- **Commit:** A permanent snapshot that moves files from the Staging Area to the Local Repository.
- Git commands
  - **git commit:** The data moves from the Staging Area into your **Local Repository**, where it is assigned a unique ID and saved in your project's history.
  - **git log**: View the git commit history





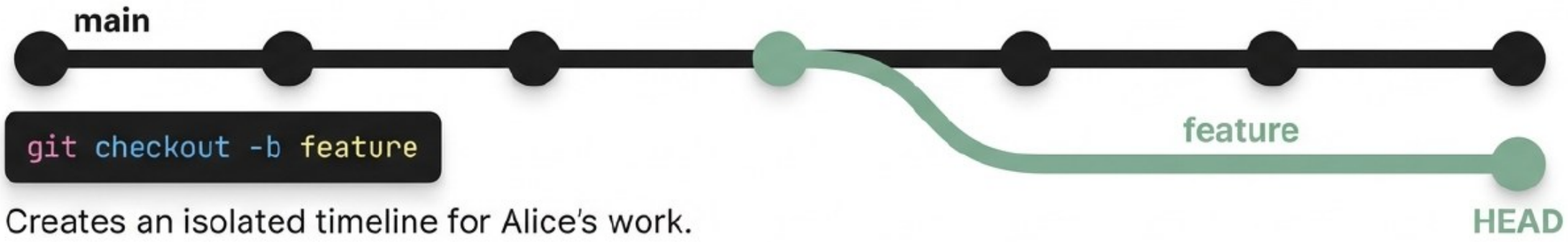
Push

- Uploads local commits from your current branch (main) to the remote server (origin/main)



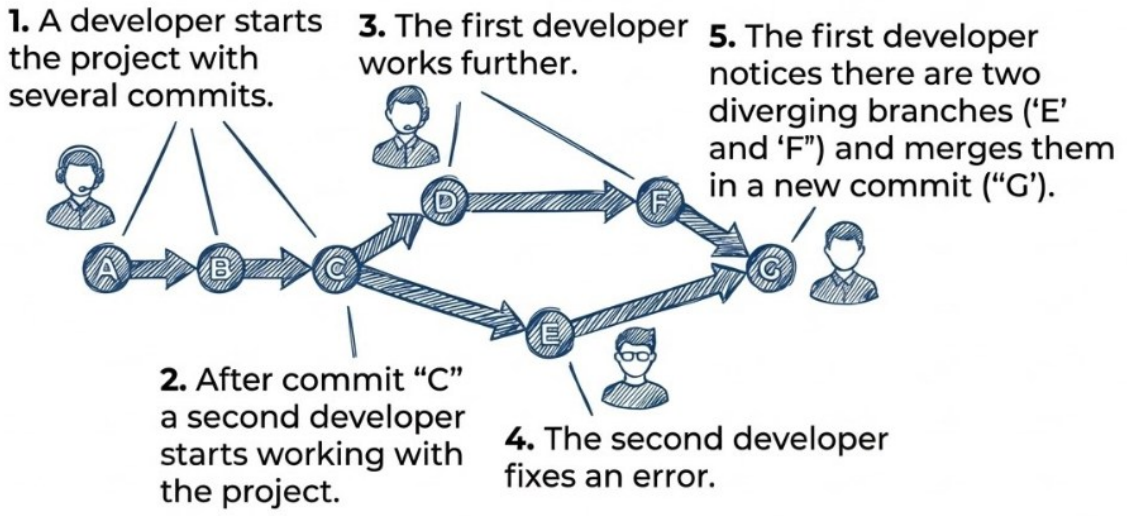
Branching

- In software development, it's common to work on features or bug fixes that are part of a larger project.
  - However, you don't want these commits in your main version history until your feature is complete.
- **Git branch:** Separate lines of development that stem from the main codebase (often referred to as the "master" or "main" branch).



Merge

- Once work on a branch is complete, use git merge to integrate it back into your main branch, combining the two branches' histories.
  - Create a new commit with two parents.
  - The resulting commit snapshot will have all of the work that has been done in both branches.
  - Illustration: https://girliemac.com/blog/2017/12/26/git-purr/

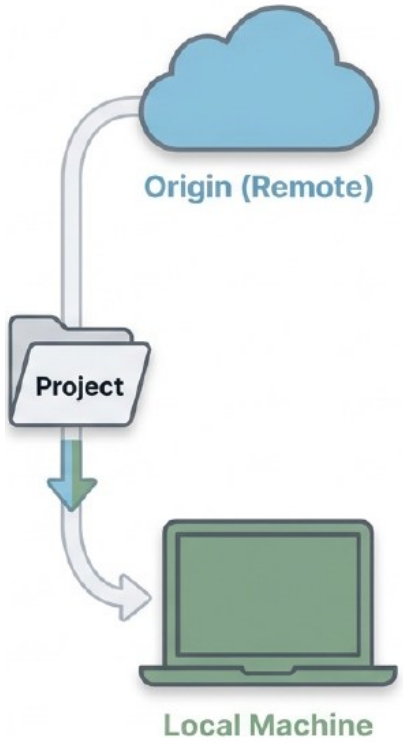


Scenario Walkthrough

- Cloning downloads the repository and its full history.
- Automatically connects your local folder to the 'origin' (GitHub)

git clone https://github.com/project/login-app.git

```
user@laptop:~$ git clone https://github.com/project/login-app.git
Cloning into 'login-app'...
remote: Enumerating objects: 3, done.
user@laptop:~$ cd login-app
user@laptop:~/login-app$ git remote -v
origin  https://github.com/project/login-app.git (fetch)
origin  https://github.com/project/login-app.git (push)
```

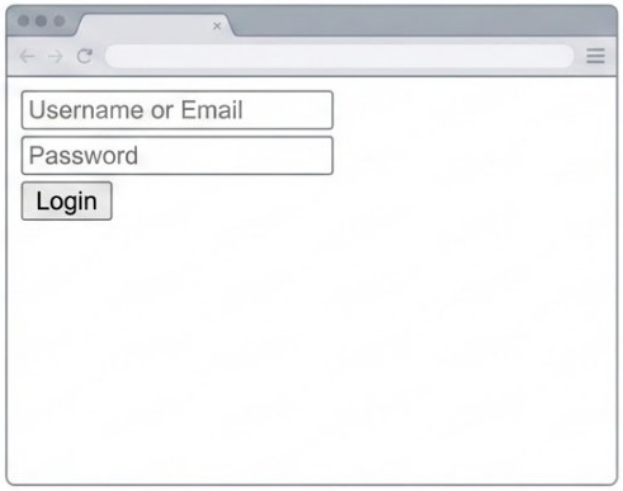


First commit: Create a login form

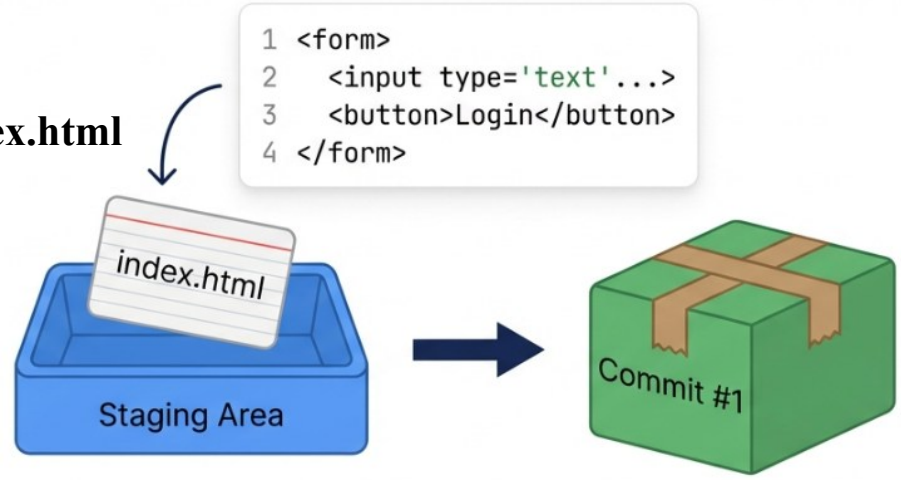
- Alice stages and commits the code change.

```
<form>
  <input type='text' placeholder='Username or Email'>
  <input type='password' placeholder='Password'>
  <button>Login</button>
</form>
```

```
~/login-app$ git add index.html
~/login-app$ git commit -m 'Add login form to index.html'
[main (root-commit) 8a2b1c] Add login form to index.html
```

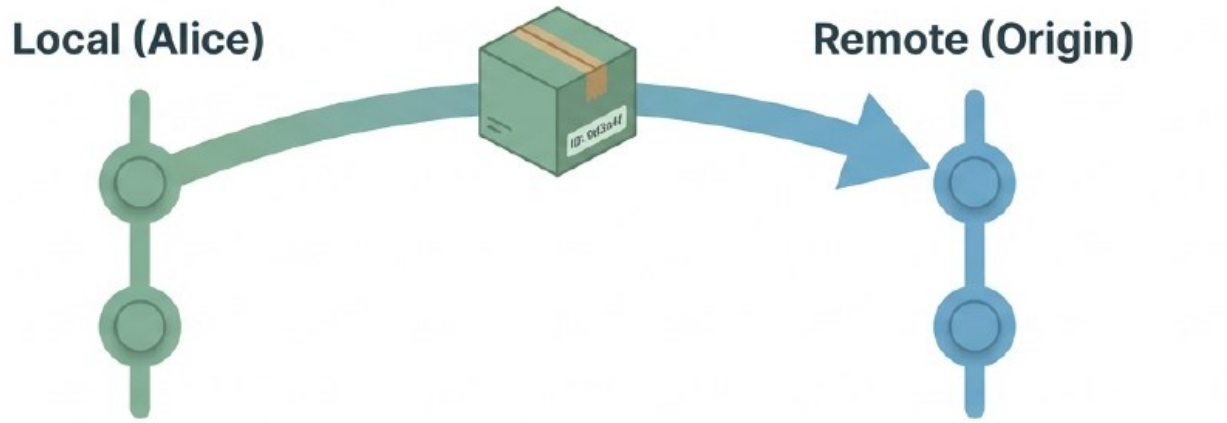


git add index.html



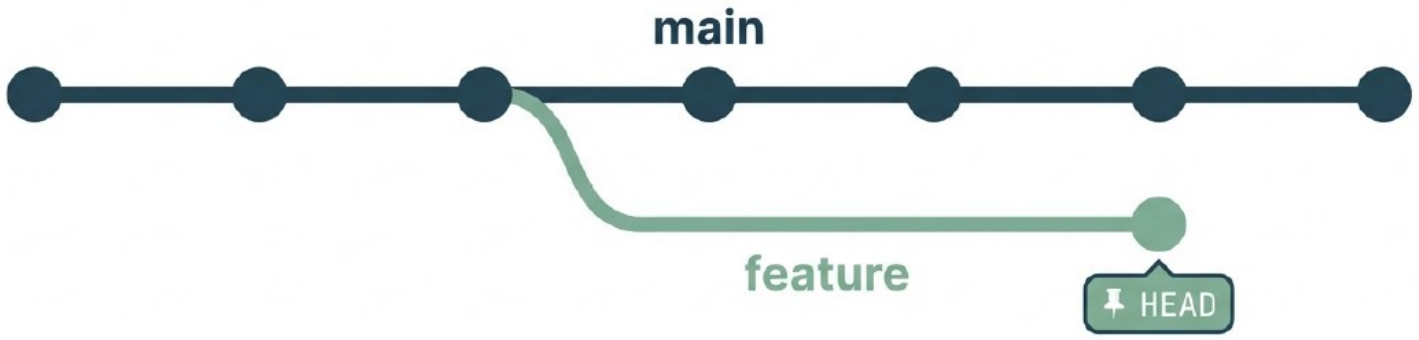
Syncing (Pushing to remote)

- **git push origin main**
  - Pushing the commits from Alice’s local branch to the server.
  - The origin now matches Alice’s local history.



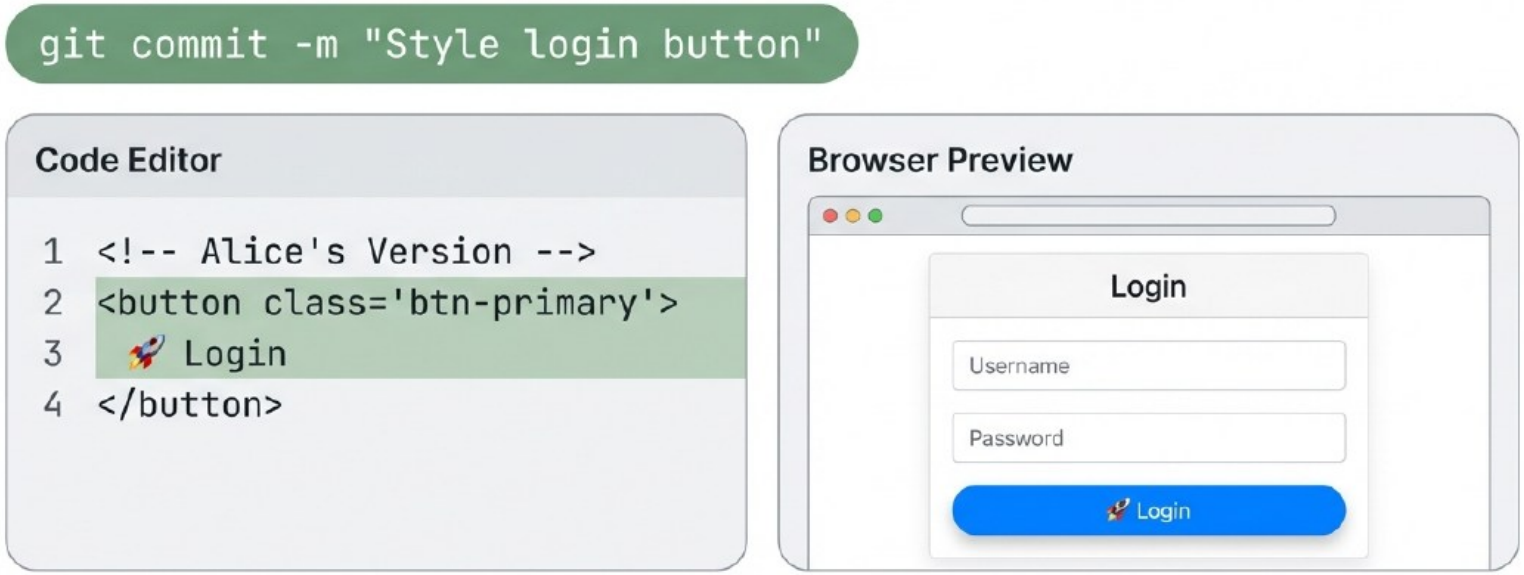
Alice create a feature branch

- Isolates Alice's new work from the main code
  - **git branch <name>:** Creates a new timeline to isolate features and bugs.
  - **git branch:** List the branches
  - **git checkout -b** creates a new branch and switches to it immediately.
  - **git checkout <name>:** Switches your active branch



Alice polish the UI and commit

- Alice style the button using CSS and add an icon to the button
- Alice stages the changes and commits locally



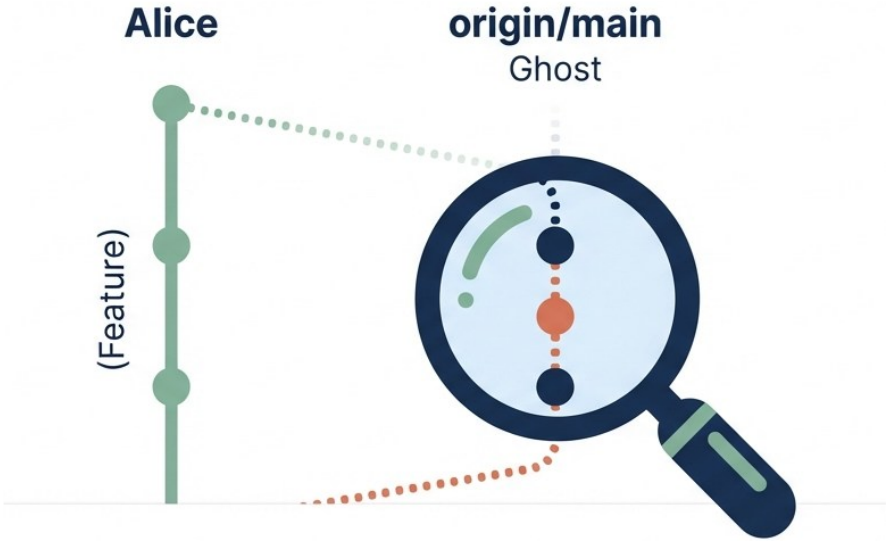
Meanwhile... Bob's Change (Remote Change)

- While Alice was working, Bob pushed changes to the main branch on the remote server.
- He added accessibility attributes and capitalized the text.
  - The aria-label attribute provides a more descriptive label for screen readers and other assistive technologies.



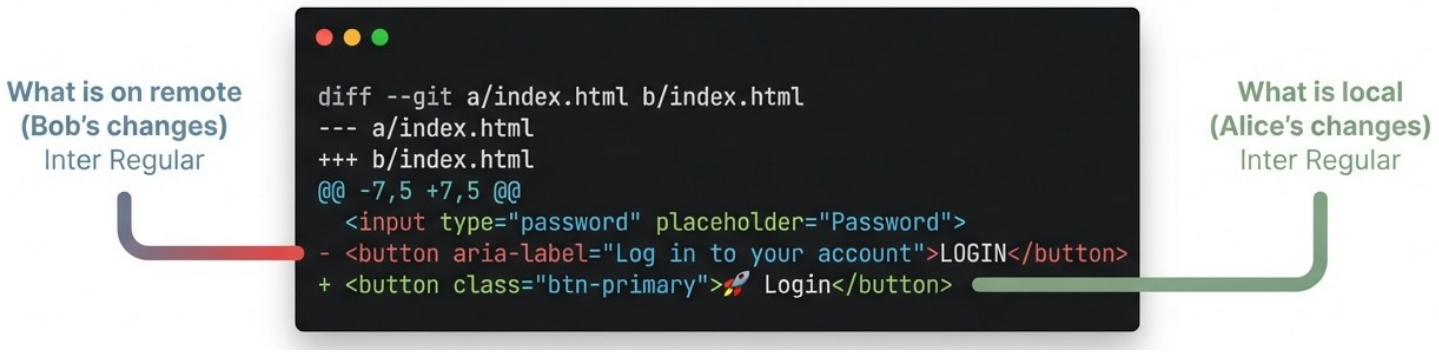
Git Fetch

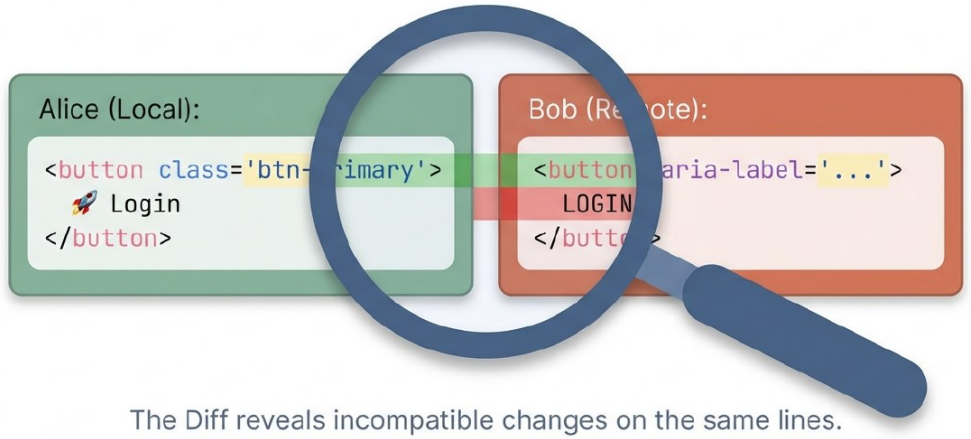
- **git fetch**
  - Updates your knowledge of the remote repository (downloading Bob's commits) but does not touch your working files yet.
- **origin/main**: Remote-tracking branch
  - A remote tracking branch in GitHub is a reference to a branch on a remote repository.
  - After you clone a repository, git will name the repository “origin” by default
  - Updated when you fetch changes from the remote repository.



Git diff

- The **git diff origin/main** command shows the difference between the current local branch (**feature**) and the remote main branch (**origin/main**)
- Bob Alice and Bob modifies the same line of code!



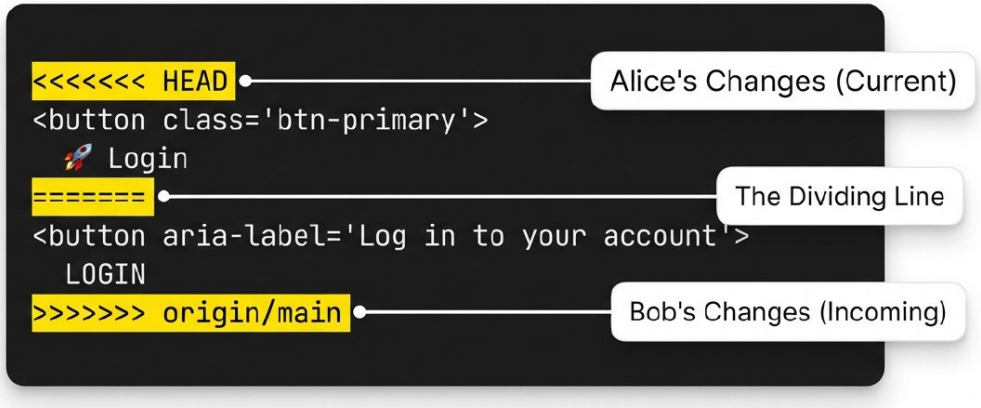


Merge

git merge origin/main

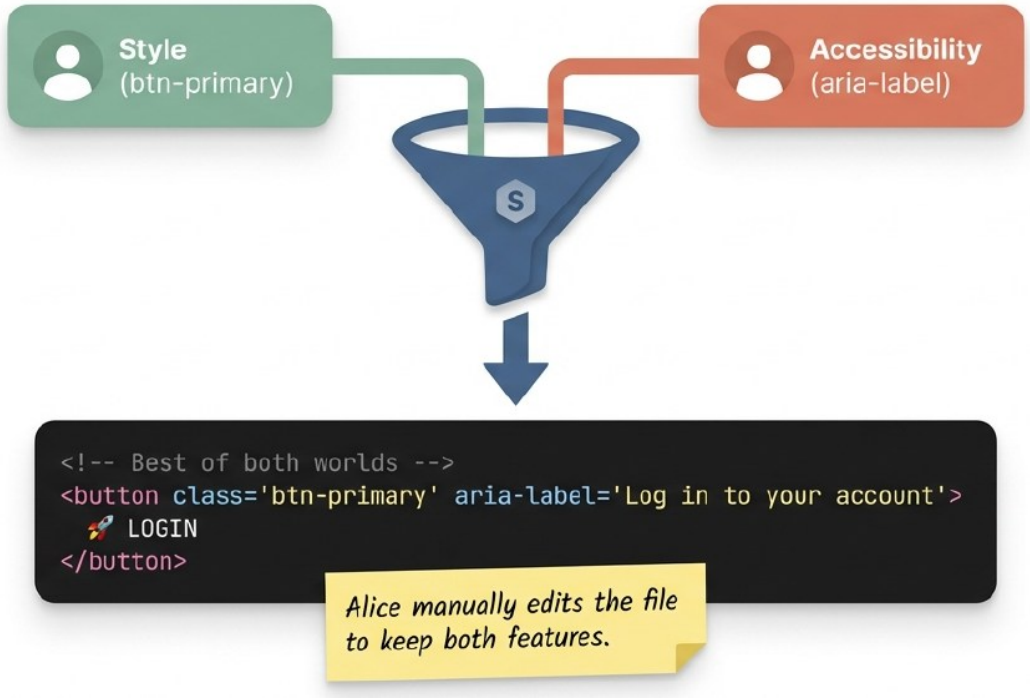
- Since both modified the same line, Git cannot automatically decide which change to keep.
- Automatic Merge Failed.
- Conflict marker: Git edits the file to show both versions simultaneously, asking Alice to choose.

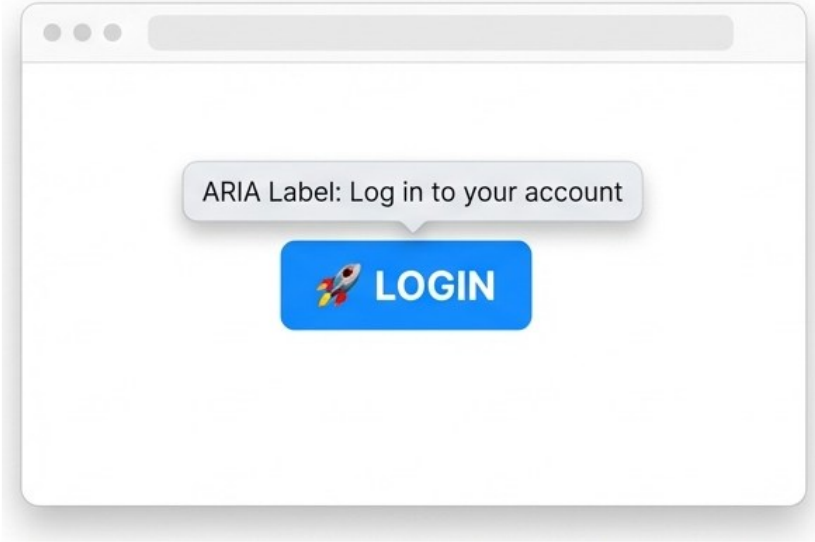
```
~/login-app$ git merge origin/main
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```



Resolving the conflict

- Alice combines the best parts of both versions
- Manually deletes the markers and combines the logic
  - Git cannot do this automatically (it requires human intent)



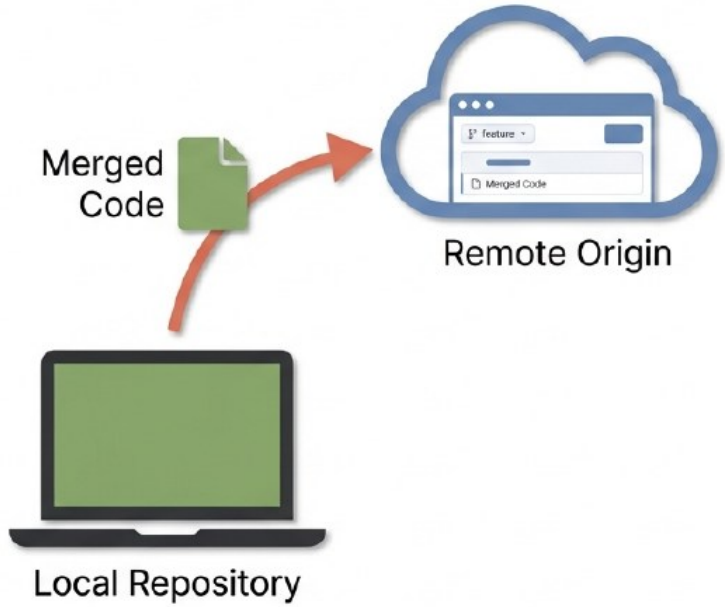


Finalizing the merge

- Alice stage the change, commit the changes, and push her feature branch to the remote repository

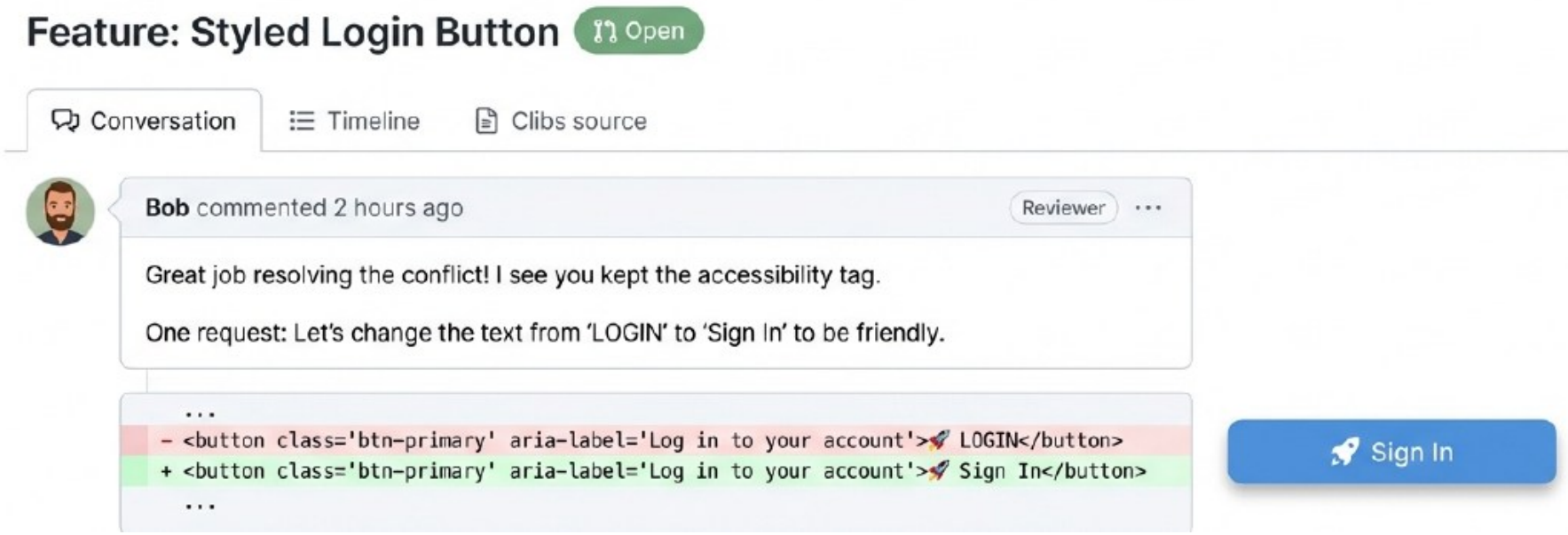
git add index.html
git commit -m 'Merge origin/main; resolve login button conflict'
git push origin feature





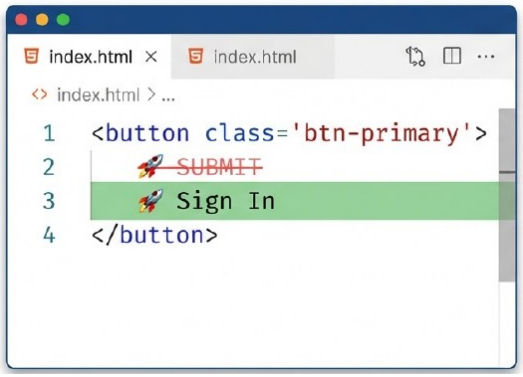
Pull request

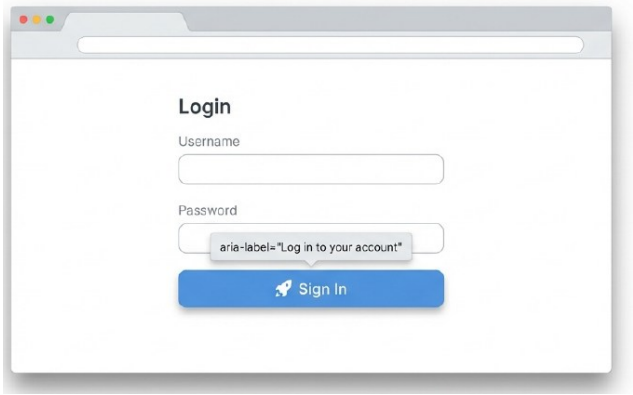
- A **Pull Request (PR)** is a dedicated space for teammates to inspect code, ask questions, and ensure quality before the work becomes permanent.
- Bob reviews the code changes and provide his comments

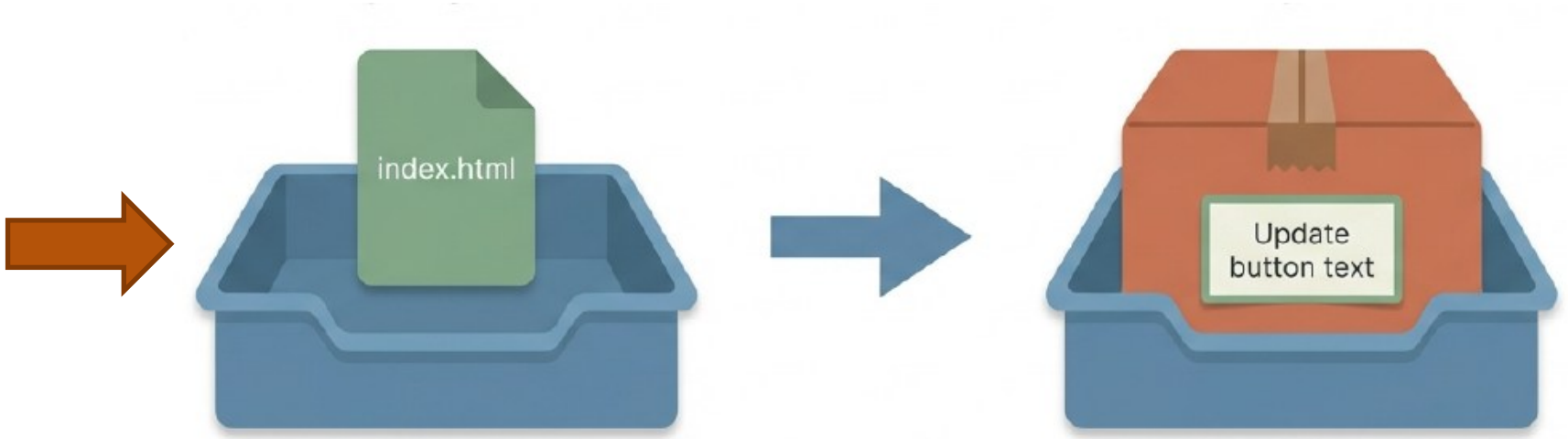


Addressing the feedback

- Alice edits the code locally, staging and committing the fix







git add index.html

git commit -m 'Update button text'

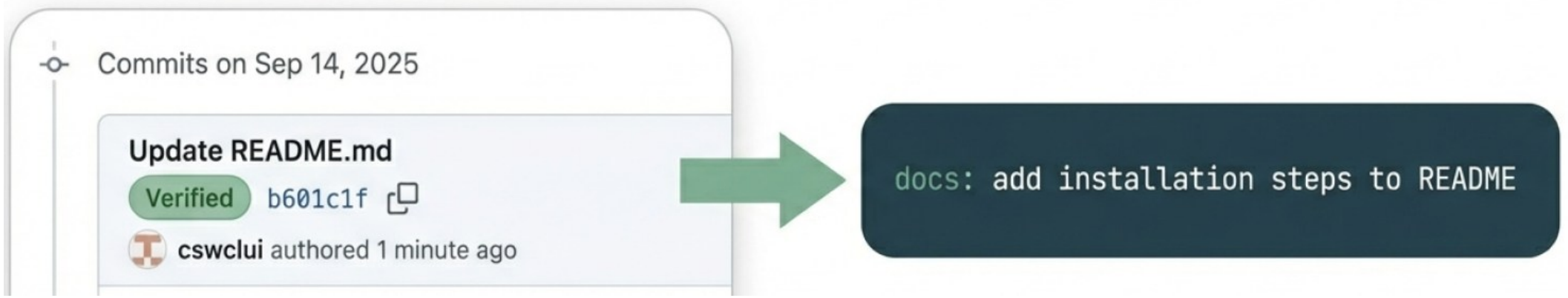
Approve and Merge

- Sync the fix to the PR: **git push origin feature**
  - You do NOT need a new Pull Request.
  - Pushing to the same branch automatically updates the existing conversation on GitHub.
- Bob approves the change and merges new features into the main branch

Conventional Commits

- A lightweight specification for structuring commit messages to create an explicit and consistent commit history.
- Common commit types include:
  - **feat:** a new feature
  - **fix:** a bug fix
  - **docs**: documentation changes
  - **style:** formatting or style changes
  - **refactor:** code refactoring without adding features or fixing bugs
  - **test**: adding or modifying tests
  - **chore:** non-feature, non-bug changes for project maintenance tasks
    - e.g. Updating dependencies in files like package.json, modify configuration files like .gitignore, changes to project documentation, etc

Examples



Chore

```
chore: configure .gitignore to
exclude .env secrets
```

Changes to build process/auxiliary tools.

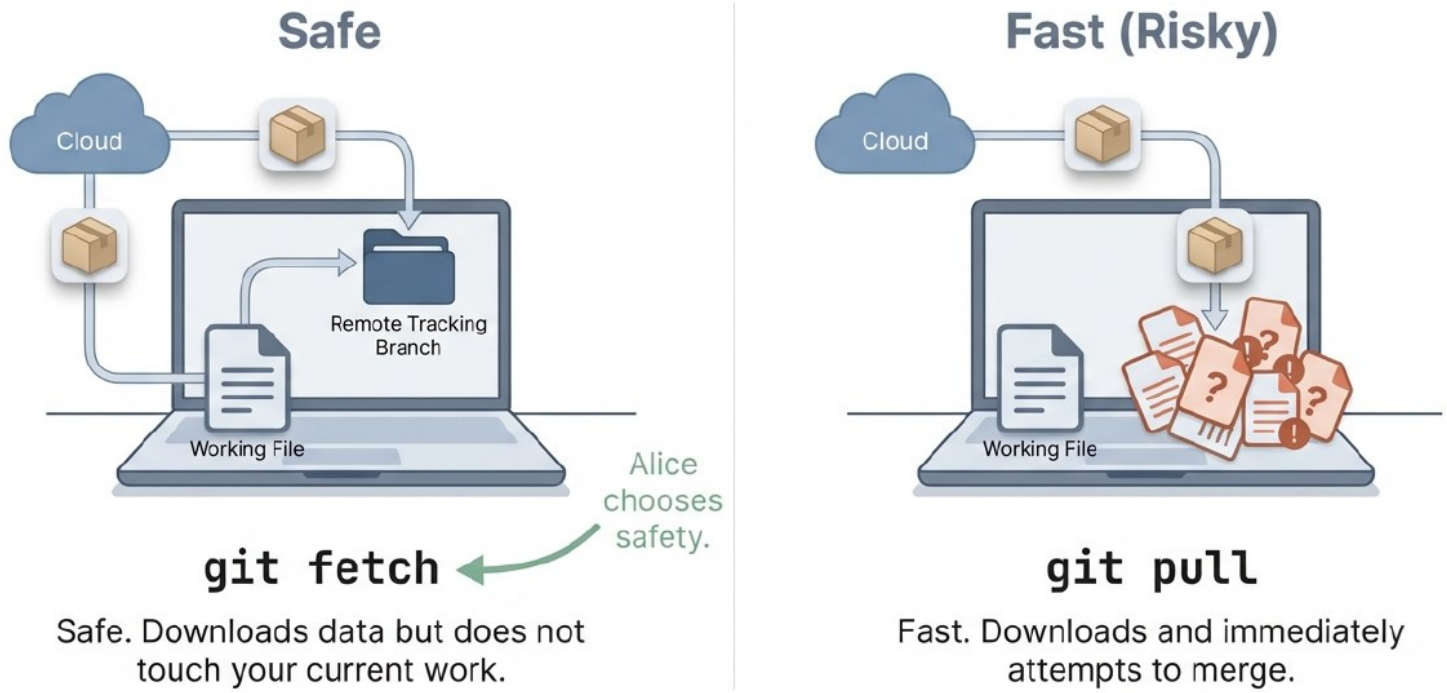
Test

```
test: add unit tests for backend API
```

Adding missing tests, no production code change.

Git fetch vs. Git Pull

- **Git Fetch:** Downloads new data but does **NOT** touch your working files.
  - It is safe and allows for review before merging.
- **Git Pull:** Downloads data **AND** immediately merges it into your current work.
  - **Git Fetch + Git Merge**



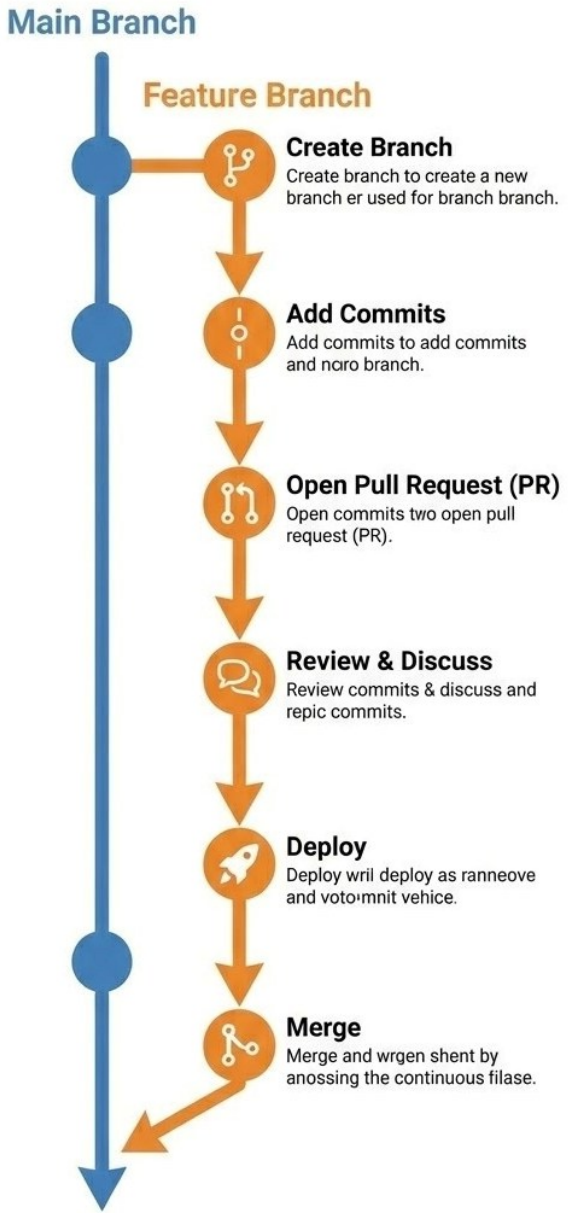
Pull in changes from main often

- When working on a feature branch, regularly pulling in changes from main helps keep your feature branch up to date and synchronized with the latest updates made by others.
- Reduces the risk of merge conflicts and makes it easier to integrate your work back into the main branch later.
  - Resolve small conflicts incrementally, rather than facing many or complex conflicts all at once when merging later

GitHub Flow

A lightweight, branch-based workflow designed for teams and projects with regular deployments.

- Create a Branch
  - Branch off from the main branch to create an isolated environment for new ideas and features.
  - Allows safe feature development and prevents conflicting changes between developers.
- Add Commits
  - Make edits and save progress by committing changes frequently to the feature branch.
  - Frequent commits provide a transparent history and make it easier to revert specific changes without undoing unrelated work.
- Open a Pull Request (PR)
  - Propose merging your branch changes back into main.
  - This starts a discussion and review process.
- Review and Discuss Changes
  - Reviewers can suggest improvements or ask questions on the PR.
  - Developers can continue to make commits to the branch to address feedback.
- Merge into Main
  - Once approved, the PR is merged into the main branch.
  - This integrates the feature branch changes into production-ready code.



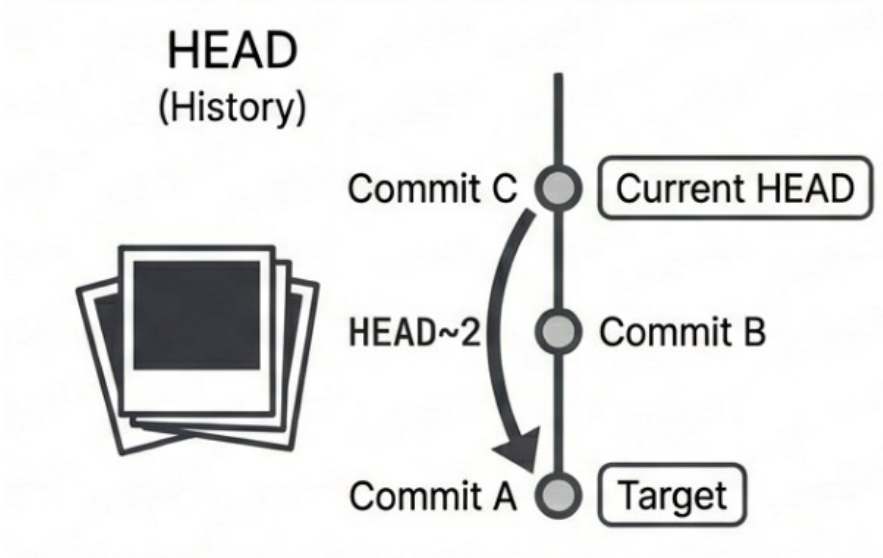
Git Resets

git reset [--mode] [commit_id]

- We have a repository with three commits. We are currently at Commit C.
- We want to ‘undo’ the last two commits (B and C) and return the repository to the state of Commit A.

git reset [mode] HEAD~2

HEAD points to the latest commit on the current branch.
HEAD~2: This refers to the commit that is two commits before the current HEAD.

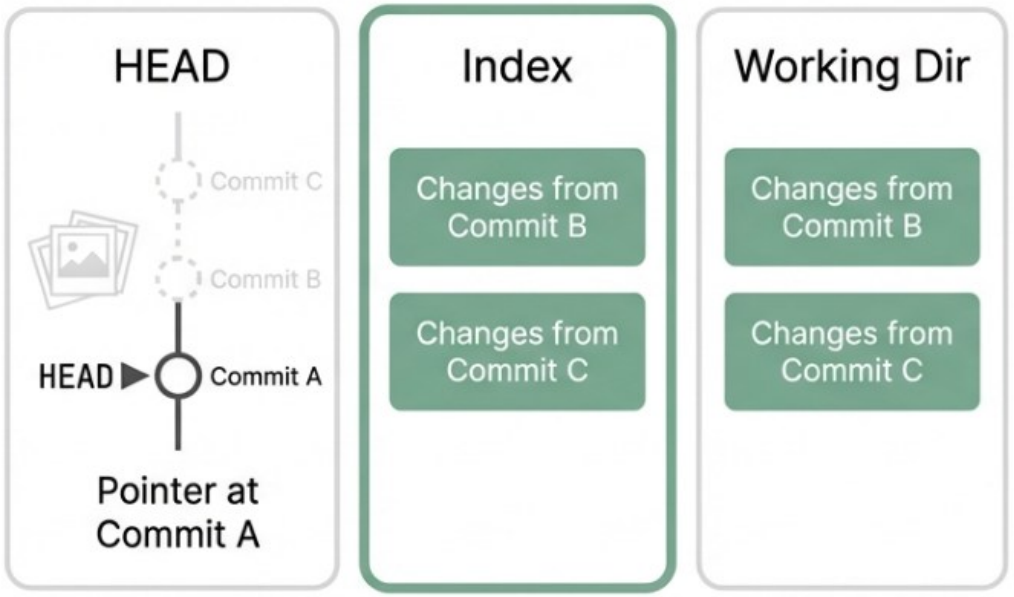


Soft Reset

"I regret that these were separate commits."

- The history rewinds, but your changes are caught in the Index (Staging Area).
  - They are already staged and ready to be committed immediately
- Use --soft when you want to group the work from multiple messy commits into one clean, polished commit. This is often called "squashing".

git reset --soft HEAD~2
git commit -m 'New clean feature'



Mixed Reset (Default)

"I regret committing, and I want to keep working."

- The history rewinds and the Index is cleared. Your changes are preserved in the Working Directory as "unstaged" modifications.
- Use this when you want to keep your code but rethink how you stage it.
  - Perhaps you included a file by mistake, or you want to split the work into entirely different commits.

git reset HEAD~2
OR
git reset --mixed HEAD~2

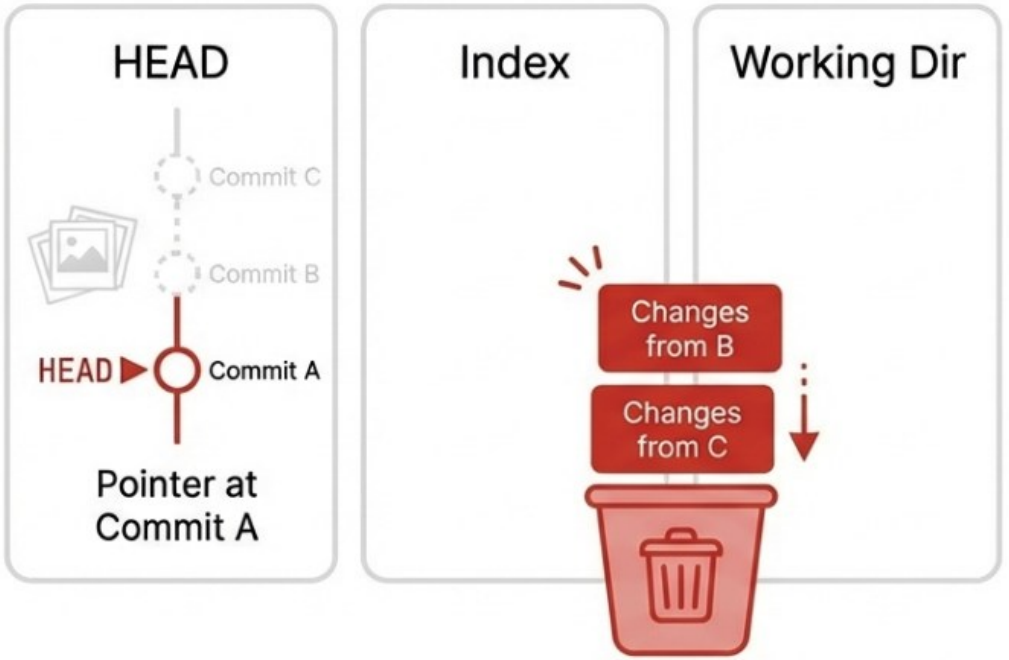


Hard Reset

- "I regret that these commits ever happened."
- The history rewinds, and ALL changes in the Index and Working Directory are destroyed. The repository now matches Commit A exactly.

Use this to scrap failed experiments. You want to obliterate recent work and return the project to the last moment things were working correctly.

Warning: This deletes uncommitted work. Always check git status first to ensure you aren't losing something valuable.



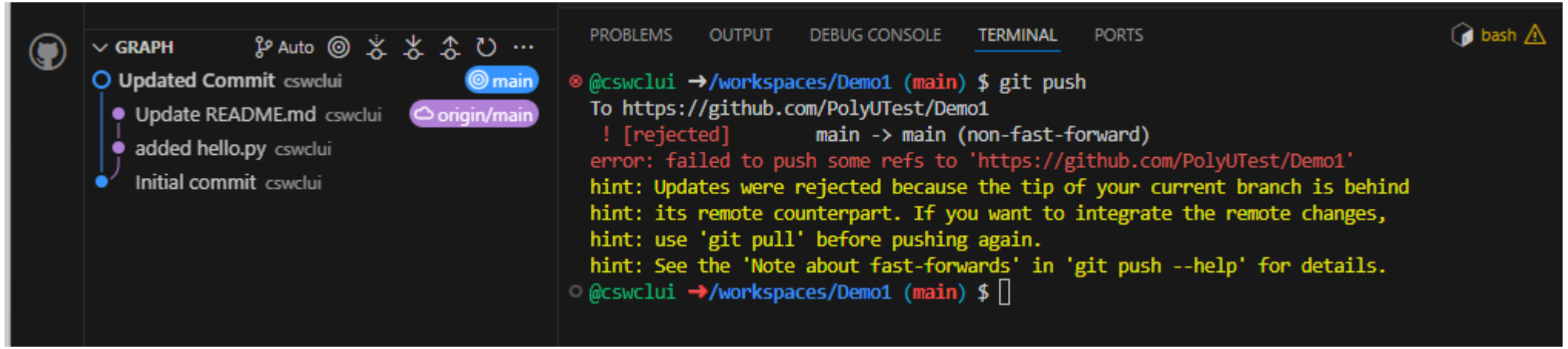
Types of Git Reset

- **git reset --soft**
  - Only moves the current branch's HEAD to the specified commit.
  - Changes since that commit will be preserved and left staged (in the index).
  - Useful for squashing commits or altering the last commit message without altering the working directory.
- **git reset –mixed** (Default)
  - Moves the current branch's HEAD to the specified commit and updates the index to match.
  - Changes since that commit in the working directory will be preserved but unstaged.
  - This is the default mode when no option is provided with git reset.
- **git reset --hard**:
  - Completely resets the working directory and index (staging area) to match the specified commit.
  - Any changes to tracked files in the working directory since that commit will be discarded.
  - **Use with caution**: this will permanently delete both staged and unstaged changes.

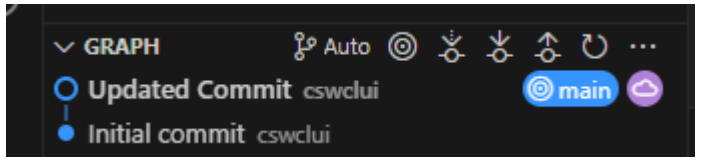
	Soft	Mixed	Hard
HEAD	Moved to Target	Moved to Target	Moved to Target
Index	STAGED (Kept)	CLEAN (Reset)	CLEAN (Reset)
Working Directory	MODIFIED (Kept)	MODIFIED (Kept)	DESTROYED

Force Push

- If you undo changes in your local branch, your branch becomes different from the remote branch.
- When you try to push, Git blocks the push to prevent losing changes on the remote.



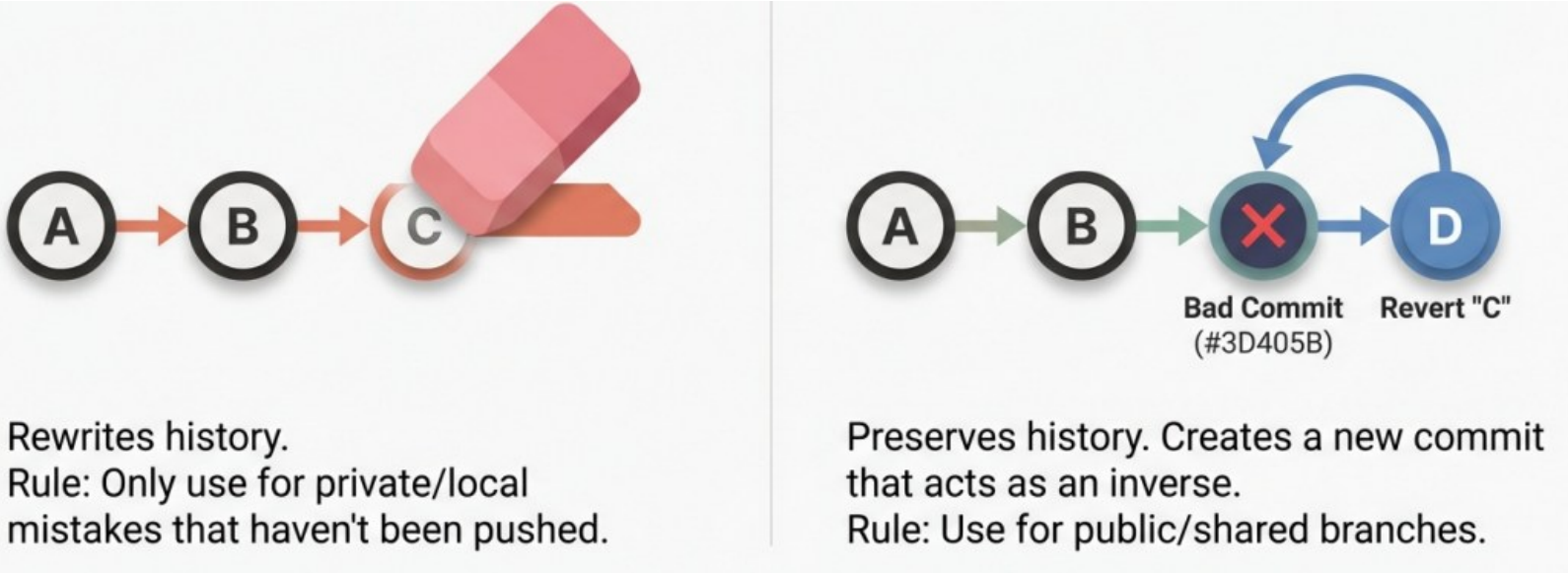
```
@cswclui ➜/workspaces/Demo1 (main) $ git push --force
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 399 bytes | 399.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/PolyUTest/Demo1
 + 8f7f3a9...f4a2806 main -> main (forced update)
```



- **git push –force**
  - Overwrite the remote branch with your local branch, replacing its history with yours.
  - Any changes on the remote that are not in your local branch will be deleted.
- Use it carefully, especially if others are working on the same branch.
  - If your teammates add new commits to the remote branch and you use git push --force, their commits will be removed from the remote branch. Their work will be lost
- Best practice
  - Never reset a branch that has already been pushed to a shared remote repository.
  - If others have pulled your branch, resetting it will cause conflicts and confusing history for everyone else.
  - Only reset commits that exist solely on your local machine.

Git Revert

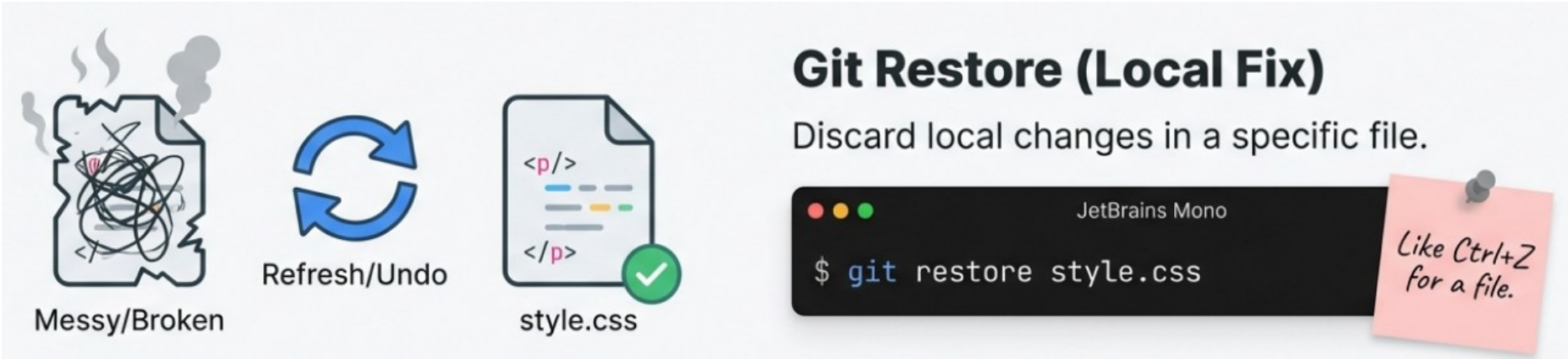
- Instead of deleting history, it creates a brand-new commit that introduces the inverse of the changes from the commit you are reverting.
  - The safest way to undo a commit, especially in a collaborative environment
- It doesn't rewrite history, so it won't cause issues for collaborators who have already pulled the original commit.



Git restore

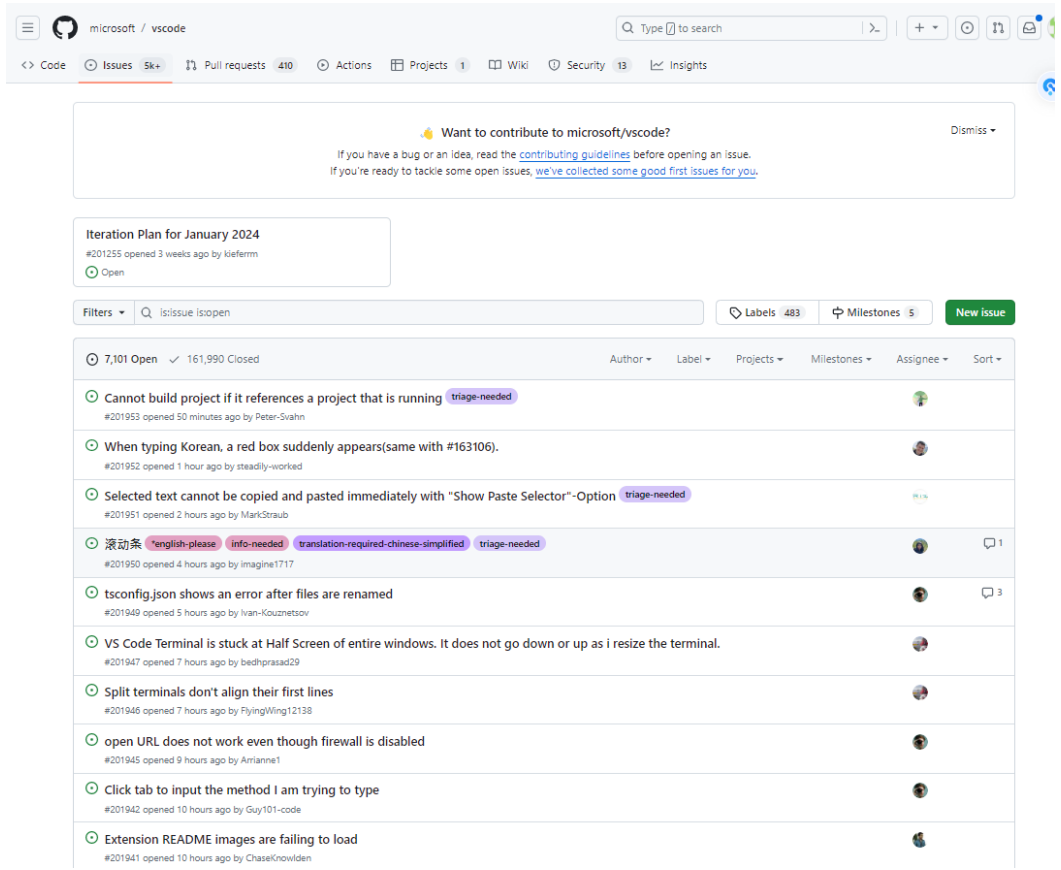
- It reverts a file (or files) in the Working Directory to match the version in the last commit (HEAD).
  - E.g. when you've made experimental changes to a file that you regret and want to quickly revert just that one file back to its last committed state.

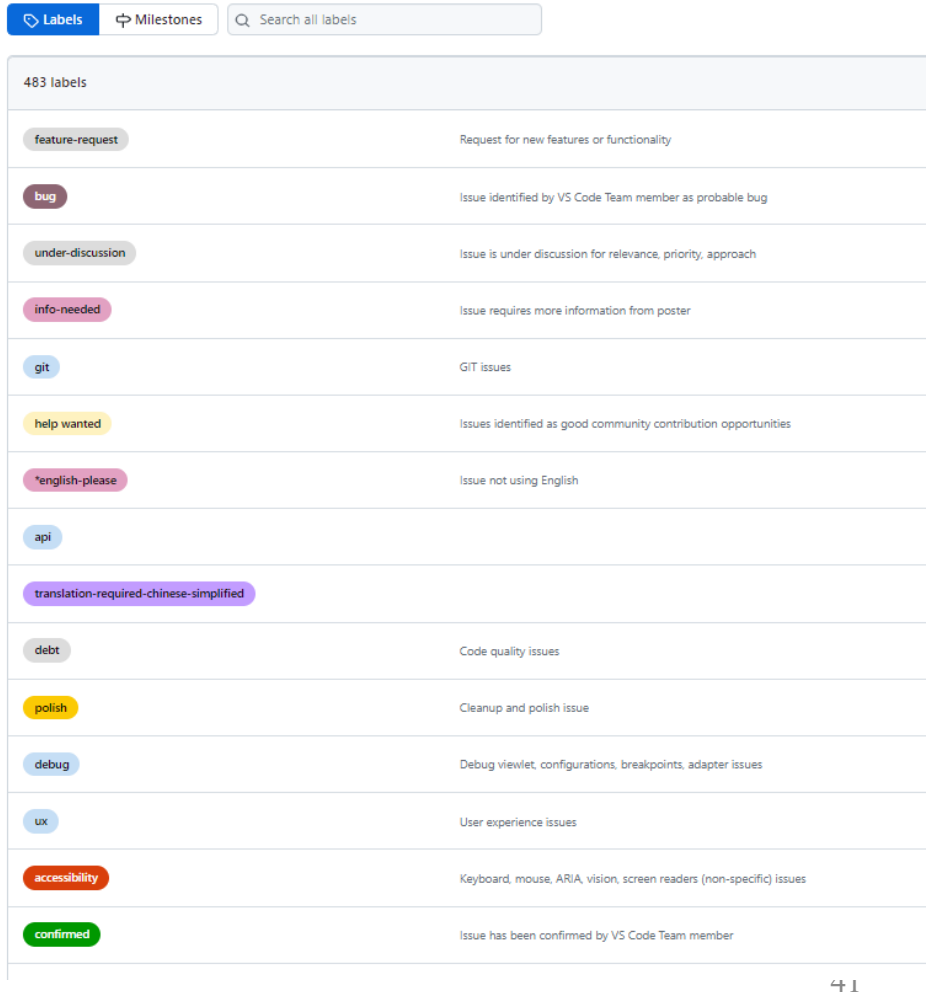
git restore style.css



GitHub Issues

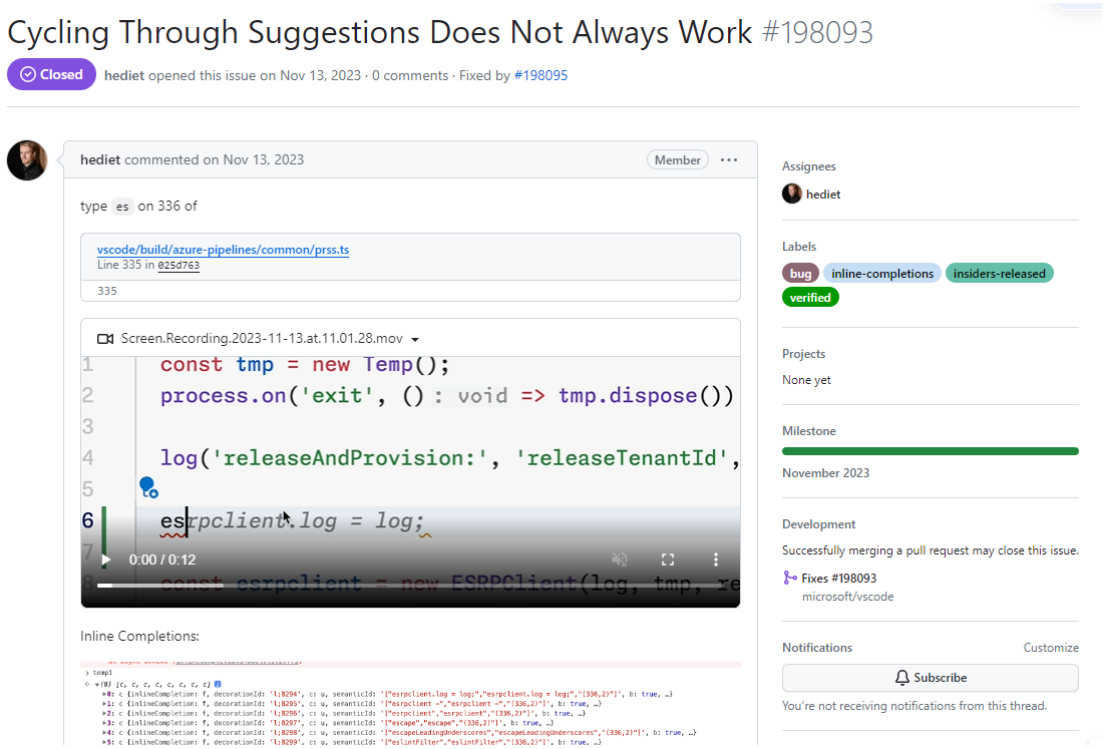
- A way to track enhancements, tasks, or bugs for your work on GitHub.



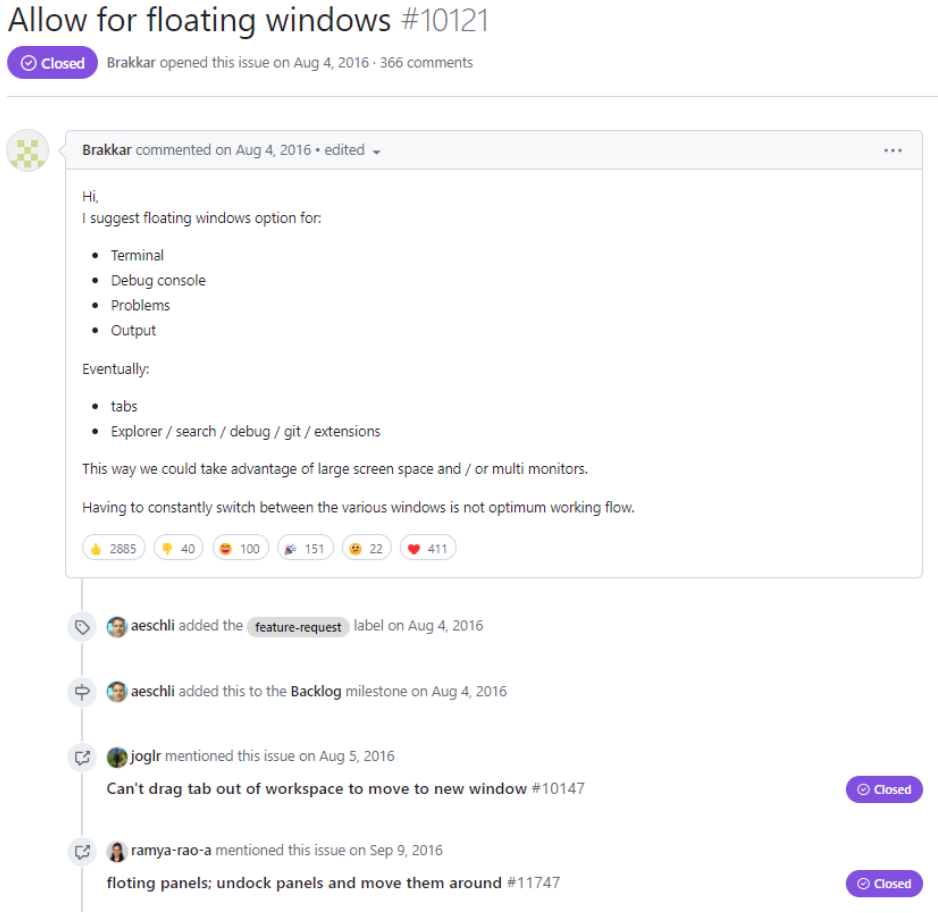


https://github.com/microsoft/vscode/issues

Example issues: Bug Report/Feature Request



https://github.com/microsoft/vscode/issues/198093



https://github.com/microsoft/vscode/issues/10121

Give me a sample template for bug report, proposing a new feature, and feature enhancement in Github issues

GPT-4 Poe

Below are sample templates for a bug report, proposing a new feature, and feature enhancement for GitHub issues. These templates can guide contributors to provide the necessary details to understand and address the issue or suggestion effectively.

Bug Report Template

markdown Copy

```markdown
## Bug Report

### Title
A clear and descriptive title for the issue to identify the bug.

### Description
A clear and concise description of what the bug is.

### Steps to Reproduce
1. Go to '...'
2. Click on '....'
3. Scroll down to '....'
4. See error

### Expected Behavior
What you expected to happen.

### Actual Behavior
What actually happened. Include screenshots or recordings if applicable.

### Possible Solution
If you have a suggestion for a fix/reason for the bug, please explain here.

### Your Environment
- Version used:
- Operating System and version:
- Link to your project:
- Browser Name and version:
- Any other relevant environment information:

### Additional Context
Add any other context about the problem here.

### Logs
If applicable, add logs to help explain your problem.
```

New Feature Proposal Template

markdown Copy

```markdown
## Feature Proposal

### Title
A clear and concise title that reflects the proposed feature.

### Summary
A brief summary of the feature.

### Motivation
Why is this feature necessary and what problems does it solve?

### Detailed Design
A detailed explanation of how this feature should work.

### Alternatives Considered
A discussion of any alternative solutions or features you've considered.

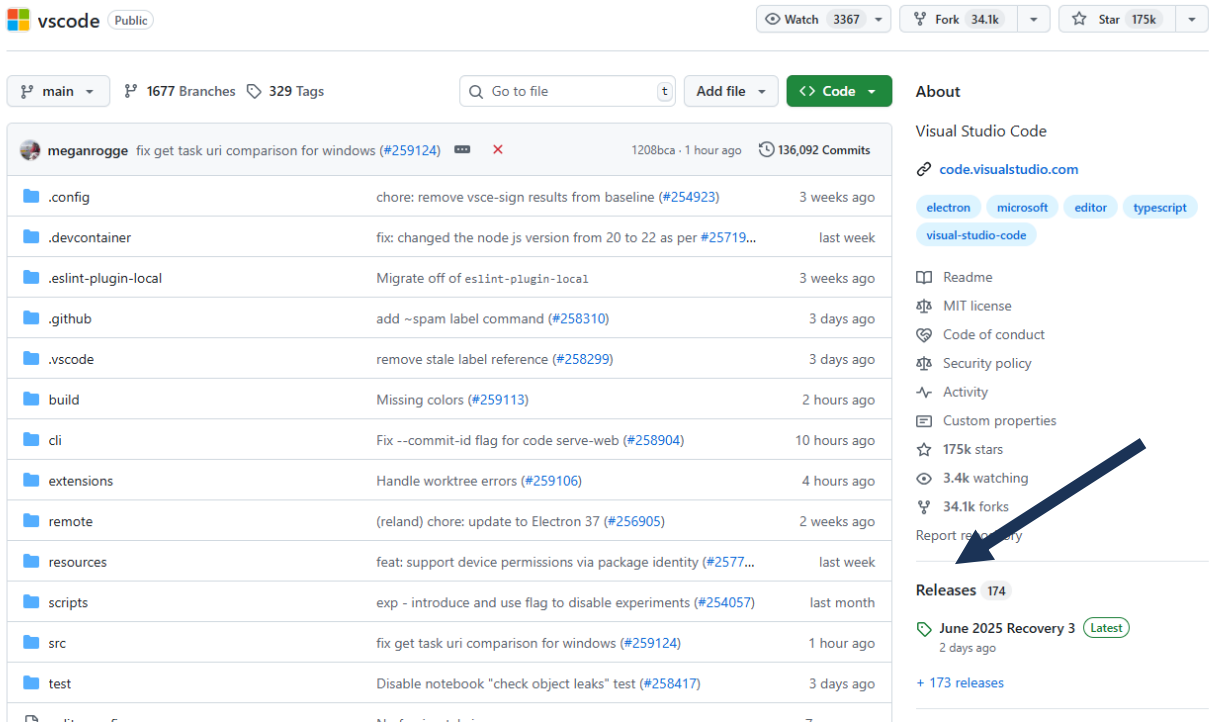
### Additional Context
Any additional information or context about the feature proposal, such as mockups or example
s.

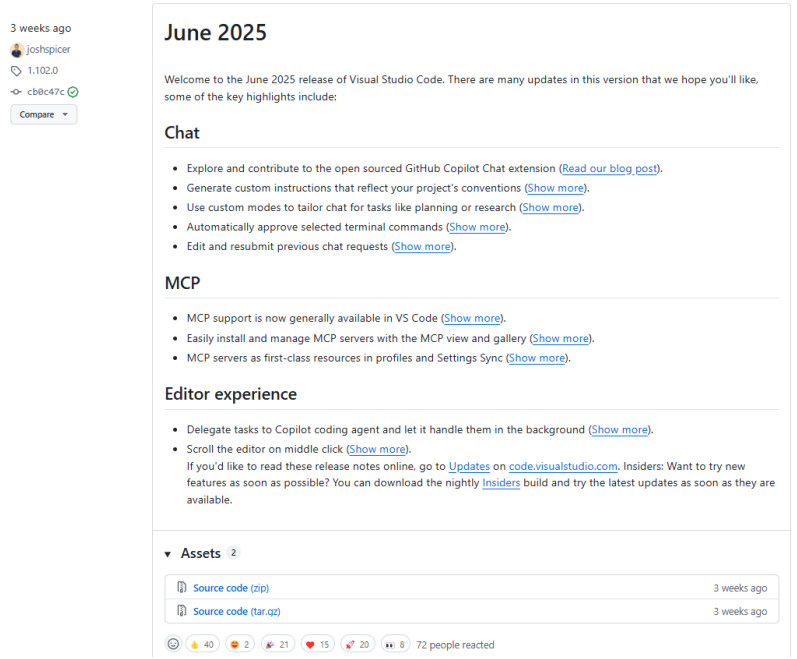
### Impact
If known, the potential impact this feature may have on existing features or systems.

### Implementation
Suggest an idea for implementing the feature, if applicable.
```

Release

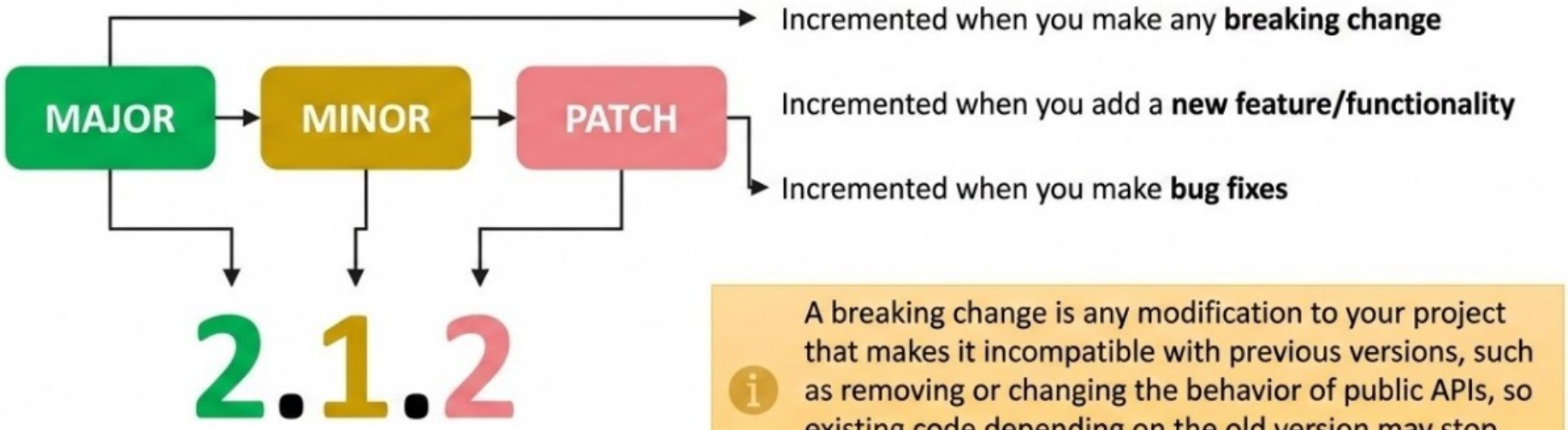
- **Release:** Creating a **release** refers to the process of packaging and distributing a specific, stable version of your software to users or other systems.
- **Release notes:** A documented summary of the changes, improvements, fixes, and important information included in a specific software release.





Semantic Versioning

- A specification, a simple set of rules that dictates how version numbers of your projects are assigned and incremented.
  - MAJOR version is incremented when you make any breaking change
  - MINOR version is incremented when you add a new feature/functionality
  - PATCH version is incremented when you make bug fixes



Exercise

Write down the version number after each commit below:

Initial version: v1.0.0

fix: remove exposed Azure API key →

feat: add Python Flask API backend →

feat: upgrade to v2 API structure →